

PHASE 0

JURISLINK V2

Rapport d'Audit du Schema Supabase

Audit complet de la base de donnees, securite, isolation multi-tenant,
stockage et traçabilité

PROJET	REFERENCE	DATE	POSTGREST
switch21/jurislink	zosqktvmihktqbbgdbgx	2025-06-18	v14.5

AVERTISSEMENT

Limitations de l'Audit

Cet audit a ete realise via l'API REST Supabase (PostgREST + Auth Admin + Storage). La connexion directe PostgreSQL etant impossible (DNS IPv6 uniquement, sandbox IPv4-only), certains elements necessitent une verification complementaire via le SQL Editor du dashboard Supabase.

ELEMENT	STATUT	DETAIL
Tables / Colonnes / Types	Complet	Via OpenAPI spec
Donnees existantes	Complet	Via REST API service_role
Auth utilisateurs	Complet	Via Auth Admin API
Storage buckets	Complet	Via Storage API
RPC fonctions (signatures)	Complet	Via OpenAPI spec
RLS politiques	Partiel	Texte SQL non accessible via REST
Triggers	Non verifiable	Necessite pg_catalog
Fonctions SQL (corps)	Non verifiable	Necessite pg_proc
Index	Non verifiable	Necessite pg_indexes
Contraintes CHECK/UNIQUE	Non verifiable	Necessite information_schema

Action requise

Pour completer l'audit (RLS, triggers, fonctions, index, contraintes), merci d'executer le script SQL fourni en Section 11 dans le SQL Editor du dashboard Supabase et fournir le resultat.

SECTION 1

Inventaire des Tables

18 tables + 2 vues

1.1 Tables metier

#	TABLE	LIGNES	CLE PRIMAIRE	TENANT_ID	ROLE METIER
1	tenants	3	id (uuid)	N/A	Cabinet juridique
2	users	8	id (uuid)	NULLABLE	Profils utilisateurs
3	clients	4	id (uuid)	NOT NULL	Clients du cabinet
4	cases	3	id (uuid)	NOT NULL	Dossiers juridiques
5	case_assignments	4	id (uuid)	NOT NULL	Avocats assignes aux dossiers
6	case_notes	2	id (uuid)	NOT NULL	Notes de dossier
7	documents	3	id (uuid)	NOT NULL	Documents stockes
8	events	5	id (uuid)	NOT NULL	Evenements / calendrier
9	event_assignments	5	id (uuid)	MANQUANT	Assignment evenements
10	tasks	0	id (uuid)	NOT NULL	Taches
11	invoices	4	id (uuid)	NOT NULL	Factures
12	messages	3	id (uuid)	NOT NULL	Messages internes
13	notifications	2	id (uuid)	NOT NULL	Notifications
14	currencies	4	id (uuid)	N/A	Devises

1.2 Tables systeme / securite

#	TABLE	LIGNES	ROLE
15	audit_logs	35	Journal d'audit avec chaine de hachage
16	audit_chain_alerts	0	Alertes d'integrite de la chaine d'audit
17	sessions_log	0	Journal des sessions
18	rate_limit_buckets	0	Rate limiting
19	csp_violations	0	Violations Content Security Policy

1.3 Vues

VUE	LIGNES	DESCRIPTION
v_active_sessions	0	Sessions actives avec info utilisateur
v_csp_violations_top	0	Top des violations CSP

SECTION 2

Detail des Colonnes par Table

Table tenants

COLONNE	TYPE	NULLABLE
id	uuid	NOT NULL (PK)
name	string	NOT NULL
logo_url	string	Oui
language	string	Oui (default 'fr')
timezone	string	Oui (default 'Europe/Paris')
phone, email, address	string	Oui
niu	string	Oui
plan	string	Oui ('starter', 'premium')
max_users	integer	Oui
max_storage_gb	number	Oui
is_active	boolean	Oui
created_at, updated_at	timestamptz	Oui

Table users

COLONNE	TYPE	NULLABLE	NOTES
id	uuid	NOT NULL	PK, FK vers auth.users
tenant_id	uuid	Oui	NULL pour root_admin
role	string	NOT NULL	Pas de contrainte ENUM
full_name, email	string	NOT NULL	
preferred_language	string	Oui	
is_active	boolean	Oui	
failed_login_attempts	integer	NOT NULL	Defaut 0
session_count_today	integer	NOT NULL	Defaut 0
locked_until	timestamptz	Oui	Verrouillage compte
last_login_at	timestamptz	Oui	
last_session_id	string	Oui	

Table clients

COLONNE	TYPE	NULLABLE
id	uuid NOT NULL (PK)	
tenant_id	uuid NOT NULL	
full_name	string NOT NULL	
company, phone, email, address, notes	string	Oui

Table cases

COLONNE	TYPE	NULLABLE	NOTES
id	uuid	NOT NULL (PK)	
tenant_id	uuid	NOT NULL	
client_id	uuid	NOT NULL	FK vers clients
title	string	NOT NULL	
description	string	Oui	
status	string	Oui	Pas de contrainte ENUM
open_date	date	Oui	
is_secret	boolean	Oui	Dossier confidentiel
payment_status, outcome	string	Oui	

Table documents

COLONNE	TYPE	NULLABLE	NOTES
id	uuid	NOT NULL (PK)	
tenant_id	uuid	NOT NULL	
case_id	uuid	Oui	FK vers cases
uploader_id	uuid	NOT NULL	FK vers users
file_name	string	NOT NULL	
file_path	string	NOT NULL	Chemin Supabase Storage
file_size	integer	NOT NULL	En bytes
tags	jsonb	Oui	Tableau de tags

Table events

COLONNE	TYPE	NULLABLE
id	uuid NOT NULL (PK)	
tenant_id	uuid NOT NULL	
case_id	uuid	Oui (FK vers cases)
title	string NOT NULL	
start_time, end_time	timestampz NOT NULL	
event_type, criticality	string	Oui
reminder_sent	boolean	Oui

Table notifications

COLONNE	TYPE	NULLABLE
id	uuid NOT NULL (PK)	
tenant_id	uuid NOT NULL	
user_id, event_id	uuid	Oui
title	string NOT NULL	
message, type	string	Oui
read	boolean	Oui

Table audit_logs

COLONNE	TYPE	NULLABLE	NOTES
id	uuid	NOT NULL (PK)	
tenant_id	uuid	NOT NULL	
user_id	uuid	Oui	
action	string	NOT NULL	INSERT, UPDATE, DELETE, etc.
entity	string	NOT NULL	Nom de la table
entity_id	string	NOT NULL	ID de la ligne
timestamp	timestampz	Oui	
previous_state, new_state	jsonb	Oui	Diff avant/apres
metadata	jsonb	Oui	
prev_hash, curr_hash	string	Oui	Chaine d'audit

SECTION 3

Donnees Existantes

3.1 Tenants (3 cabinets)

NOM	PLAN	UTILISATEURS	REGION
EKOKA LAW FIRM	starter	3	Douala
SCP NDOKI & ASSOCIES	premium	1	Douala
SCP MBOCK & PINDY	premium	4	Douala

3.2 Utilisateurs (8)

EMAIL	ROLE	TENANT	ACTIF	MFA
pat.epee@gmail.com	root_admin	(global)	Oui	Non
pod126@yahoo.fr	firm_admin	EKOKA	Oui	Non
charlenendoki91@gmail.com	lawyer	EKOKA	Oui	Non
thewarehouse.cm@gmail.com	secretary	EKOKA	Oui	Non
pindy@scpmock-pindy.com	firm_admin	MBOCK & PINDY	Oui	Non
mbock@scpmock-pindy.com	firm_admin	MBOCK & PINDY	Oui	Non
eboukijean@gmail.com	lawyer	MBOCK & PINDY	Oui	Non
cndoki@scp-ndoki.com	firm_admin	SCP NDOKI	Oui	Non

3.3 Distribution des donnees par tenant

TABLE	EKOKA	SCP NDOKI	MBOCK & PINDY	TOTAL
Cases	1	0	2	3
Clients	1	0	3	4
Documents	1	0	2	3
Events	2	0	3	5
Invoices	2	0	2	4
Messages	2	0	1	3
Notifications	2	0	0	2
Audit Logs	1	1	33	35

SECTION 4

Fonctions RPC (14)

4.1 Securite & Sessions

FONCTION	PARAMETRES	ROLE
get_tenant_id	(aucun, utilise auth.uid())	Retourne le tenant_id de l'utilisateur
get_user_role	(aucun, utilise auth.uid())	Retourne le role de l'utilisateur
is_aal2	(aucun, utilise auth.uid())	Verifie si MFA actif (AAL2)
is_account_locked	p_user_id	Verifie si un compte est verrouille
is_session_suspicious	p_user_id	Detecte sessions suspectes
is_case_assignee	check_case_id	Verifie si utilisateur assigne a un dossier
register_successful_login	p_user_id, p_session_id?	Enregistre connexion reussie
register_failed_login	p_email	Enregistre echec de connexion
register_session_end	p_session_id, p_reason?	Enregistre fin de session
revoke_session	p_session_id, p_revoked_by?	Revoque une session

4.2 Audit

FONCTION	PARAMETRES	ROLE
compute_audit_hash	p_action, p_entity, p_entity_id, p_metadata, p_prev_hash, p_timestamp, p_user_id	Calcule le hash pour la chaine d'audit
verify_audit_chain	p_from_id?, p_to_id?	Verifie l'integrite de la chaine

4.3 Donnees (RGPD)

FONCTION	PARAMETRES	ROLE
export_user_data	p_user_id	Exporte les donnees utilisateur (RGPD)
soft_delete_user_data	p_user_id, p_deleted_by?, p_reason?	Suppression soft (RGPD)

4.4 Systeme

FONCTION	PARAMETRES	ROLE
detect_suspicious_sessions	p_hours_back?	Detecte sessions suspectes
purge_old_csp_violations	p_max_age_days?	Nettoie anciennes violations CSP

SECTION 5

Storage

Buckets

BUCKET	PUBLIC	FILE SIZE LIMIT	NOTES
documents	Prive	Aucune	Documents juridiques
Logos	Public	Aucune	Logos des cabinets

Problemes Storage

- **Policies 404** : L'API Storage retourne 404 sur les policies. Aucune policy definie confirmee ou endpoint modifie.
- **Bucket logos PUBLIC** : Les logos sont accessibles sans authentication. Acceptable si seuls des logos y sont stockes.
- **Pas de file_size_limit** : Aucun des deux buckets n'a de limite de taille de fichier.

SECTION 6

Auth & MFA

6.1 Statut Auth

8 utilisateurs, tous avec email confirme automatiquement a la creation.

6.2 MFA — CRITIQUE

METRIC	VALEUR
Utilisateurs avec MFA	0 / 8 (0%)
Facteurs TOTP enregistres	0
Niveau AAL	AAL1 (aucun MFA) pour tous

Risque majeur

L'application a l'infrastructure pour MFA (fonction is_aal2, mentions dans instructions.md Phase 2), mais **aucun utilisateur n'a active MFA**. Pour un outil juridique manipulant des donnees sensibles, c'est un risque majeur.

SECTION 7

Analyse Multi-Tenant

7.1 Isolation par tenant_id

TABLE	TENANT_ID	NOT NULL	STATUT
tenants	—	—	N/A (table racine)
users	Oui	Non (NULLABLE)	root_admin a NULL
clients, cases, case_assignments, case_notes	Oui	Oui	Bon
documents, events, tasks, invoices	Oui	Oui	Bon
messages, notifications, audit_logs	Oui	Oui	Bon
event_assignments	MANQUANT	—	CRITIQUE
sessions_log	Oui	Non (NULLABLE)	A surveiller
currencies	—	—	N/A (donnees globales)

7.2 Faille critique : event_assignments sans tenant_id

Faille d'isolation multi-tenant

La table event_assignments lie un evenement a un utilisateur mais **n'a pas de colonne tenant_id**. Cela signifie :

- Impossible d'appliquer une RLS basee sur tenant
- Un utilisateur du tenant A pourrait theoriquement voir les assignments d'evenements du tenant B
- La jointure avec events.tenant_id est necessaire pour filtrer, mais la RLS ne peut pas faire ca directement

Action : Ajouter tenant_id uuid NOT NULL REFERENCES tenants(id) + index (Migration 001).

SECTION 8

Chaine d'Audit — Analyse

La table audit_logs possede une excellente structure avec prev_hash / curr_hash pour la chaine de hachage, previous_state / new_state pour le diff, et metadata pour les infos supplementaires. Les fonctions compute_audit_hash et verify_audit_chain sont egalement presentes.

8.2 Problemes identifies

PROBLEME	SEVERITE	DETAIL
Tous les hashes sont NULL	Critique	Les 35 entrees ont prev_hash=null et curr_hash=null. La chaine d'integrite n'est pas utilisee.
user_id NULL	Moyen	Certaines entrees (creation tenant, creation user) ont user_id=null.
Pas d'IP	Moyen	Pas de colonne ip_address dans audit_logs
Pas de user_agent	Moyen	Pas de colonne user_agent dans audit_logs
Naming incoherent	Faible	La colonne entity sert de resource_type, mais le naming ne correspond pas aux instructions.md

Comparaison avec instructions.md — Ecart

9.1 Elements compatibles (reutilisables)

ELEMENT	STATUT	COMMENTAIRE
Table tenants	Bon	Bonne structure
Table users	Bon	Avec securite (locked_until, failed_login_attempts)
Tables clients, cases, documents, events, invoices	Bon	Structure adequate
Table audit_logs	Bon	Excellente structure avec chaine de hachage
14 fonctions RPC	Bon	Couvrant securite, audit, sessions
2 buckets Storage	Bon	Documents (private) + Logos (public)
8 utilisateurs + 3 tenants	Bon	Donnees de demo preservees

9.2 Colonnes manquantes (Migration ADD COLUMN)

TABLE	COLONNE	TYPE	PHASE
event_assignments	tenant_id	uuid NOT NULL	CRITIQUE (Phase 3)
cases	reference	varchar(50)	Phase 9
cases	case_type	varchar(100)	Phase 9
cases	priority	varchar(20)	Phase 9
cases	assigned_lawyer_id	uuid FK	Phase 9
cases	next_deadline	timestampz	Phase 9
clients	responsible_lawyer_id	uuid FK	Phase 8
clients	status	varchar(20)	Phase 8
clients	last_activity_at	timestampz	Phase 8
notifications	category	varchar(50)	Phase 12
notifications	resource_type	varchar(50)	Phase 12
notifications	resource_id	uuid	Phase 12
audit_logs	ip_address	varchar(45)	Phase 11
audit_logs	user_agent	text	Phase 11
documents	mime_type	varchar(100)	Phase 10
documents	version	integer	Phase 10
documents	is_confidential	boolean	Phase 20

9.3 Nouvelles entites necessaires

ENTITE	JUSTIFICATION	PHASE
document_versions	Historique de versionnage des documents	Phase 10

9.4 Contraintes manquantes

TABLE	CONTRAINTE CHECK	VALEURS AUTORISEES
users.role	chk_user_role	root_admin, firm_admin, lawyer, secretary, collaborator, accountant, trainee
cases.status	chk_case_status	new, open, in_progress, pending, closed, archived
invoices.status	chk_invoice_status	draft, sent, paid, overdue, cancelled
events.criticality	chk_event_criticality	low, normal, high, urgent

9.5 Relations manquantes

RELATION	TYPE	JUSTIFICATION
event_assignments.tenant_id → tenants.id	FK + index	CRITIQUE — Isolation multi-tenant
cases.assigned_lawyer_id → users.id	FK	Avocat responsable (Phase 9)
clients.responsible_lawyer_id → users.id	FK	Avocat responsable (Phase 8)

9.6 Index recommandes

TABLE	COLONNES	REQUETE TYPIQUE
users	(tenant_id, role)	Liste utilisateurs par cabinet et role
cases	(tenant_id, status)	Filtre dossiers par statut
cases	(tenant_id, client_id)	Dossiers d'un client
clients	(tenant_id, full_name)	Recherche clients
documents	(tenant_id, case_id)	Documents d'un dossier
events	(tenant_id, start_time)	Evenements par date
notifications	(user_id, read)	Notifications non lues
audit_logs	(tenant_id, timestamp)	Historique d'audit
audit_logs	(entity, entity_id)	Suivi d'une entite
messages	(tenant_id, receiver_id, read_status)	Messages non lus
sessions_log	(user_id, started_at)	Historique sessions

Plan de Migrations (non-destructives)

Les 4 migrations suivantes sont **entièrement non-destructives**. Elles préservent toutes les données existantes et utilisent uniquement des instructions ADD, CREATE, ALTER compatibles avec les données en place.

Migration 001 — Correction critique : tenant_id sur event_assignments

```
ALTER TABLE event_assignments
ADD COLUMN tenant_id uuid NOT NULL DEFAULT
  '00000000-0000-0000-0000-000000000000'::uuid;

UPDATE event_assignments ea
SET tenant_id = e.tenant_id
FROM events e WHERE ea.event_id = e.id;

ALTER TABLE event_assignments ALTER COLUMN tenant_id DROP DEFAULT;
ALTER TABLE event_assignments
ADD CONSTRAINT fk_event_assignments_tenant
FOREIGN KEY (tenant_id) REFERENCES tenants(id);
CREATE INDEX idx_event_assignments_tenant_id
ON event_assignments(tenant_id);
```

Migration 002 — Contraintes CHECK

```
ALTER TABLE users ADD CONSTRAINT chk_user_role
CHECK (role IN ('root_admin','firm_admin','lawyer',
  'secretary','collaborator','accountant','trainee'));

ALTER TABLE cases ADD CONSTRAINT chk_case_status
CHECK (status IN ('new','open','in_progress',
  'pending','closed','archived'));

ALTER TABLE invoices ADD CONSTRAINT chk_invoice_status
CHECK (status IN ('draft','sent','paid',
  'overdue','cancelled'));

ALTER TABLE events ADD CONSTRAINT chk_event_criticality
CHECK (criticality IN ('low','normal','high','urgent'));
```

Migration 003 — Colonnes manquantes pour V2

```

-- Cases
ALTER TABLE cases ADD COLUMN IF NOT EXISTS reference varchar(50);
ALTER TABLE cases ADD COLUMN IF NOT EXISTS case_type varchar(100);
ALTER TABLE cases ADD COLUMN IF NOT EXISTS priority varchar(20) DEFAULT 'normal';
ALTER TABLE cases ADD COLUMN IF NOT EXISTS assigned_lawyer_id uuid REFERENCES users(id);
ALTER TABLE cases ADD COLUMN IF NOT EXISTS next_deadline timestampz;

-- Clients
ALTER TABLE clients ADD COLUMN IF NOT EXISTS responsible_lawyer_id uuid REFERENCES users(id);
ALTER TABLE clients ADD COLUMN IF NOT EXISTS status varchar(20) DEFAULT 'active';
ALTER TABLE clients ADD COLUMN IF NOT EXISTS last_activity_at timestampz;

-- Notifications
ALTER TABLE notifications ADD COLUMN IF NOT EXISTS category varchar(50);
ALTER TABLE notifications ADD COLUMN IF NOT EXISTS resource_type varchar(50);
ALTER TABLE notifications ADD COLUMN IF NOT EXISTS resource_id uuid;

-- Audit logs
ALTER TABLE audit_logs ADD COLUMN IF NOT EXISTS ip_address varchar(45);
ALTER TABLE audit_logs ADD COLUMN IF NOT EXISTS user_agent text;

-- Documents
ALTER TABLE documents ADD COLUMN IF NOT EXISTS mime_type varchar(100);
ALTER TABLE documents ADD COLUMN IF NOT EXISTS version integer DEFAULT 1;
ALTER TABLE documents ADD COLUMN IF NOT EXISTS is_confidential boolean DEFAULT false;

```

Migration 004 — Index performance

```

CREATE INDEX IF NOT EXISTS idx_users_tenant_role ON users(tenant_id, role);
CREATE INDEX IF NOT EXISTS idx_cases_tenant_status ON cases(tenant_id, status);
CREATE INDEX IF NOT EXISTS idx_cases_tenant_client ON cases(tenant_id, client_id);
CREATE INDEX IF NOT EXISTS idx_clients_tenant_name ON clients(tenant_id, full_name);
CREATE INDEX IF NOT EXISTS idx_documents_tenant_case ON documents(tenant_id, case_id);
CREATE INDEX IF NOT EXISTS idx_events_tenant_start ON events(tenant_id, start_time);
CREATE INDEX IF NOT EXISTS idx_notifications_user_read ON notifications(user_id, read);
CREATE INDEX IF NOT EXISTS idx_audit_logs_tenant_ts ON audit_logs(tenant_id, timestamp);
CREATE INDEX IF NOT EXISTS idx_audit_logs_entity ON audit_logs(entity, entity_id);
CREATE INDEX IF NOT EXISTS idx_messages_receiver_read ON messages(tenant_id, receiver_id, read_status);
CREATE INDEX IF NOT EXISTS idx_sessions_log_user ON sessions_log(user_id, started_at);

```

SECTION 11

Script SQL à Executer dans le Dashboard

Pour compléter l'audit (RLS, triggers, fonctions, index, contraintes existantes), veuillez exécuter ce script dans le **SQL Editor** du Supabase Dashboard et fournir le résultat :

```

-- RLS STATUS
SELECT relname AS table_name, relrowsecurity AS rls_enabled,
       relforcerowsecurity AS rls_forced
FROM pg_class
WHERE relnamespace = 'public'::regnamespace
AND relkind = 'r'
ORDER BY relname;

-- RLS POLICIES
SELECT schemaname, tablename, policyname, permissive, roles,
       cmd, qual, with_check
FROM pg_policies
WHERE schemaname = 'public'
ORDER BY tablename, policyname;

-- TRIGGERS
SELECT event_object_table AS table_name, trigger_name,
       event_manipulation, action_timing, action_statement
FROM information_schema.triggers
WHERE trigger_schema = 'public'
ORDER BY event_object_table, trigger_name;

-- FUNCTIONS (public schema)
SELECT p.proname AS name,
       pg_get_function_arguments(p.oid) AS args,
       pg_get_function_result(p.oid) AS returns,
       prosrc AS source
FROM pg_proc p
JOIN pg_namespace n ON p.pronamespace = n.oid
WHERE n.nspname = 'public'
ORDER BY p.proname;

-- INDEXES
SELECT schemaname, tablename, indexname, indexdef
FROM pg_indexes
WHERE schemaname = 'public'
ORDER BY tablename, indexname;

-- CONSTRAINTS (UNIQUE + CHECK)
SELECT tc.table_name, tc.constraint_name,
       tc.constraint_type, cc.check_clause
FROM information_schema.table_constraints tc
LEFT JOIN information_schema.check_constraints cc
  ON tc.constraint_name = cc.constraint_name
WHERE tc.table_schema = 'public'
AND tc.constraint_type IN ('UNIQUE', 'CHECK')
ORDER BY tc.table_name;

-- ENUM TYPES
SELECT t.typname, e.enumlabel, e.enumsortorder
FROM pg_type t JOIN pg_enum e ON t.oid = e.enumtypid
ORDER BY t.typname, e.enumsortorder;

-- STORAGE POLICIES
SELECT * FROM storage.policies ORDER BY bucket_id, name;

```

Resume Executif

Forces identifiees

ARCHITECTURE MULTI-TENANT

En place

CHAINE D'AUDIT

Structure OK

FONCTIONS RPC

14 fonctions

DONNEES DE DEMO

Preservees

Le schema existant presente une architecture solide avec isolation multi-tenant, un systeme d'audit complet avec chaine de hachage, 14 fonctions RPC couvrant securite, audit et sessions, ainsi que des tables de securite (sessions_log, rate_limit_buckets, csp_violations). L'isolation des documents par tenant dans le Storage est egalement une bonne pratique en place.

Faiblesses critiques

5 points critiques a corriger

1

event_assignments sans tenant_id — Faille d'isolation multi-tenant permettant un acces cross-tenant theorique

2

Chaine de hachage d'audit non activee — Tous les 35 hashes sont NULL, l'integrite n'est pas verifiable

3

MFA a 0% d'adoption — Aucun des 8 utilisateurs n'a active l'authentification multi-facteurs

4

RLS policies non verifiables — Confirmation par SQL requise (script Section 11)

5

Storage policies non verifiables — API retourne 404, verification SQL requise

Ameliorations prioritaires

#	ACTION	MIGRATION	PRIORITE
1	Ajouter tenant_id a event_assignments	001	CRITIQUE
2	Verifier et renforcer les RLS	Script S11	CRITIQUE
3	Activer la chaine de hachage d'audit	Backfill + trigger	HAUTE
4	Ajouter les 17 colonnes manquantes	003	MOYENNE
5	Ajouter les contraintes CHECK	002	MOYENNE
6	Creer les 11 index manquants	004	MOYENNE
7	Definir les Storage policies	A determiner	MOYENNE

Rapport genere automatiquement — Les migrations proposees sont non-destructives et preservent toutes les donnees existantes.
Jurislink V2 — Phase 0 Audit — 2025-06-18